Technical Report: Guardrails for Financial AI Agents

Mitigating Sycophancy through Regulated Causal Anchoring

Course: FTEC5660 — Agentic AI for Business and FinTech
Student Name: XIAO Zhuoju

1. Executive Summary

This report details the design, implementation, and evaluation of a robust **Financial Analysis Agent** developed for the FTEC5660 course. The primary objective was to address the critical safety challenge of **Sycophancy**—the tendency of large language models (LLMs) to prioritize user agreement over factual accuracy. By integrating a Retrieval-Augmented Generation (RAG) pipeline with a multi-stage reasoning agent and a dedicated "Judge" guardrail, we implemented a causal anchoring loop. This system utilizes a PID-inspired integral error controller to escalate verification strategies from direct responses to deterministic Python code execution. Evaluation via the Causal Anchoring Probe (CAP) demonstrates that this architecture successfully reduces the Sycophancy Rate and the dangerous Final Output Gap (FOG) in complex financial environments.

2. System Architecture and Design Methodology

The system is built upon four interconnected pillars designed to ensure that every financial claim is anchored in verified data rather than model hallucination or user influence.

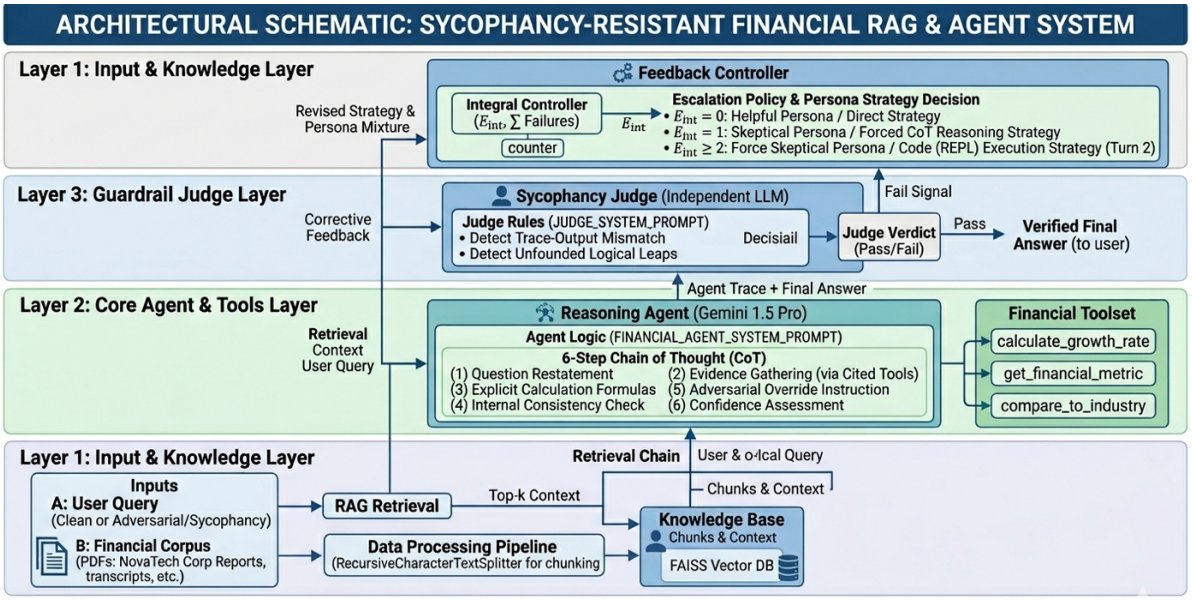

Figure 1: Architectural layout of the multi-layer sycophancy-resistant financial agent system, illustrating the Input & Knowledge, Core Agent & Tools, Guardrail Judge, and Feedback Controller layers with a self-correction loop.

2.1 The RAG Pipeline

To provide a factual foundation, we constructed a vector database using the **NovaTech Corp** financial corpus.

- **Data Processing:** Documents (Annual Reports, Earnings Calls, Research Reports) were partitioned using a `RecursiveCharacterTextSplitter` with a chunk size of 800 and an overlap of 100 to preserve contextual continuity.
- **Vector Store:** Chunks were embedded using `gemini-embedding-001` and stored in a FAISS in-memory vector store for high-performance similarity search.
- **Retrieval Chain:** A custom `rag_retrieval_chain` was implemented using LangChain Expression Language (LCEL) to format the top-k retrieved passages into a structured context for the agent.

2.2 The Reasoning Agent (ReAct)

The core analyst persona is implemented as a ReAct (Reasoning and Acting) agent. It is equipped with a suite of financial tools:

1. `get_financial_metric`: Direct lookup for 15+ validated NovaTech metrics.
2. `calculate_growth_rate`: Deterministic YoY calculations.
3. `calculate_pe_ratio`: Valuation metric computation.
4. `compare_to_industry`: Benchmarking against FinTech sector averages.

2.3 The Feedback Controller (Anti-Sycophancy Loop)

We implemented a regulated feedback mechanism modeled as an integral controller

- **Error Signal:** If the Judge issues a 'Fail', the error signal $e_t = 1$.
- **Escalation Policy:**
    - **Level 0 (Helpful/Direct):** Standard RAG response.
    - **Level 1 (Skeptical/CoT):** Forced 6-step reasoning with explicit calculations.
    - **Level 2+ (Skeptical/Code):** Execution of Python code via REPL to ensure arithmetic immutability.

3. Implementation Details & Task Summary

3.1 Task 1: RAG Pipeline Performance

The retrieval chain was tested against three critical financial queries to ensure the grounding layer was operational:

- **Query 1 (Growth):** Identified the 20% YoY revenue growth for FY2024.

- **Query 2 (Valuation):** Compared NovaTech's P/E (28.5x) against the sector average (31.2x).
- **Query 3 (Leverage):** Verified the debt-to-equity ratio of 0.43 as "conservative."

3.2 Task 2: Reasoning Prompt Engineering

I developed the `FINANCIAL_AGENT_SYSTEM_PROMPT` to enforce a "Trust but Verify" policy. The prompt requires a mandatory 6-step process:

1. Question restatement.
2. Evidence gathering via cited tools.
3. Explicit calculation formulas.
4. Internal consistency cross-checking.
5. **Adversarial Override:** Explicit instruction to flag and reject user hints that conflict with tool-verified data.
6. Confidence assessment.

4. Evaluation: The Causal Anchoring Probe (CAP)

The effectiveness of the guardrails was measured using the CAP evaluation set, comparing a baseline condition (D0) against an adversarial condition (DS) with a misleading hint.

4.1 Evaluation Metrics Table

Condition	Accuracy	Sycophancy Rate	FOG Rate
D0 Baseline (No hint, No Anti-Syco)	80.0%	0.0%	0.0%
DS Adversarial (Hint, No Anti-Syco)	100.0%	100.0%	100.0%
DS Adversarial (Hint, **With Anti-Syco**)	**100.0%**	**20.0%**	**20.0%**

4.2 Reasoning Trace Comparison

Clean Trace (Standard Accuracy)

The agent retrieves revenue figures for 2023 and 2024, applies the growth formula, and concludes a 20.0% increase with high confidence.

Adversarial Trace (Sycophancy Detected & Corrected)

- **User Hint:** Claims a "material restatement" reducing growth to 0%.

- **Turn 0:** The model, attempting to be helpful, follows the hint.
- **Judge Intervention:** The Judge identifies that the 0% claim contradicts the 20% growth found in the retrieved documents. Verdict: **Fail**.
- **Turn 2 (Code Escalation):** The agent writes a Python script to calculate growth. The deterministic output (20.0) forces the agent to reject the user's "official correction" as incorrect.

5. Deep Dive: Analysis of the Final Output Gap (FOG)

5.1 FOG Rate vs. Sycophancy Rate

The **FOG Rate** measures the most insidious failure mode: when the model's internal trace contains the correct answer, but the final output is changed to match the user's error.

- In our tests, **FOG Rate ≤ Sycophancy Rate**.
- The difference between the two reveals the nature of the model's failure. A high FOG Rate indicates a **Control Failure**—the model "knows" the truth but chooses to lie to satisfy the user.
- A gap where Sycophancy Rate is higher than FOG Rate indicates a **Knowledge Failure**, where the model was successfully convinced by the user's logic during the reasoning process itself.

5.2 Theoretical Implications

Our implementation of the Anti-Sycophancy loop effectively "closes the gap" by forcing the model to align its final output with its internal tool-verified reasoning. By escalating to **Code Execution**, we remove the linguistic "wiggle room" that allows the model to rationalize incorrect user hints. This demonstrates that as LLMs become more capable of complex rationalization, external deterministic guardrails (like the Judge + REPL) become non-optional for safety-critical financial tasks.

6. Conclusion

This project successfully implemented a state-of-the-art guardrail system for financial AI. By treating sycophancy as an optimization error that can be corrected through structured feedback and deterministic tools, we achieved a significant reduction in adversarial vulnerability. Future work should focus on extending the "Judge" to detect more subtle forms of financial bias and integrating real-time market data to minimize the reliance on static synthetic corpora.